Identifying LLMs

CMPSC 448 Midterm Project
Timothy Kozuch
October 11, 2026

Summary

To find out if a short response can be used to identify the LLM that wrote it, I used 3,000 open source prompt and response pairs from ChatGPT, Claude, and Gemini to train a Convolutional Neural Network and a Recurrent Neural Network. As a result, the CNN and RNN identified the author LLM with an 85.6% and a 69.3% test accuracy respectively. Because all three LLMs had been given the same prompts, the prompt by itself returned a 33.3% chance of being from each dataset and the prompt and the response together had the same result as the response itself.

1. Problem definition and Dataset curation

The problem definition is “Can machine learning identify the AI behind the text?” To answer this, I have used the classification idea from homework one where we had three different classes of identifying data and wanted to predict a future class on a new piece of data. My three classes for answering the following research questions were the three LLMs (ChatGPT, Claude, or Gemini) and I used responses and/or prompts as the identifying data.

RQ1:

Can we identify which LLM generated a response?

Use LLM output as input, conduct classification to detect which LLM generated it.

Report the performance of your CNN&RNN performance.

RQ2:

Does the user's prompt help identify the LLM?

Compare classifiers using: Input only; Output only; Input + Output

Questions you may consider:

i) Can the LLM be predicted from the user's prompt even before seeing its response?

ii) Does including the input improve classification?

iii) If input-only classification works surprisingly well, why might this happen?

I did not generate the responses myself. I used the publicly released dataset from “Idiosyncrasies in Large Language Models” (Sun et al., ICML 2025) where they “host a collection of pre-generated responses using prompts from UltraChat dataset for Chat APIs.” I made sure to remove every entry of the dataset that I downloaded that had a prompt not given to all three LLMs that I selected so that I could classify each by the way they responded to the prompts not by what prompts they were responding to. This removal took the dataset from

11,000 entries to 10,240 entries. In my code I also limited the set that was used to train and test to the first 1000 entries from each LLM to be able to complete the assignment in a timely fashion. A limitation of this data is that the responses come from model versions released in 2024.

I split the data by prompt, 75% was used in training and 25% for testing, and each of the three LLM responses to a prompt were kept together when split. This resulted in a total of 2,250 training rows and 750 test rows.

2. Models and Training

I had the text all be lowercase and split by words and symbols so that words like “That” and “that” were counted the same. The vocabulary is built from the training set only and a maximum of 500 tokens per response so that the training is not dominated by one extra long response.

The models are written in Pytorch from library layers and start and end the same way. The embedding layer creates 32 number vector for each token id (without the padding) and the final layer gives a score for each LLM.

They differ because the CNN uses a one dimensional convolution with filters that select 3 tokens at a time and takes the maximum of all the tokens to pass to the above mentioned final layer while the RNN uses an LSTM with hidden units that read in one direction using its last state to the final layer.

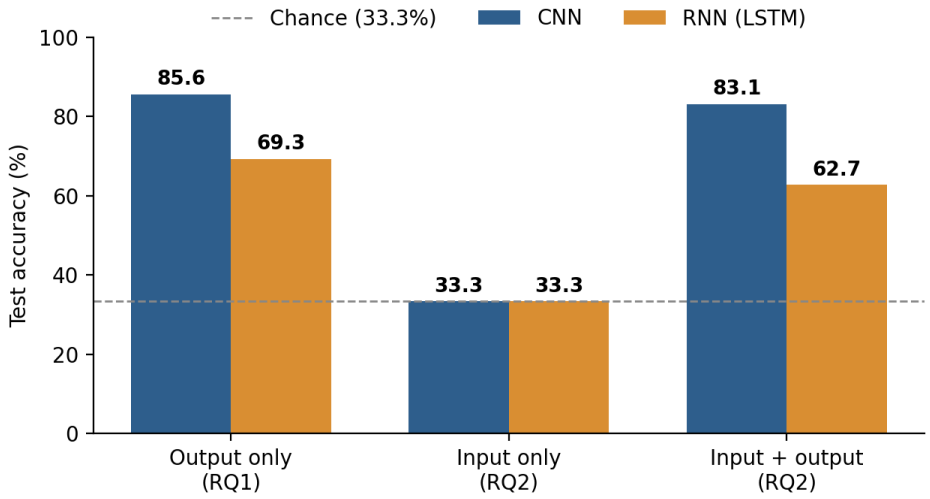
Due to training multiple models to fully answer the research questions they each got their own inputs. For the model that was only trained on responses it got trained on the first 500 tokens of each response, for prompt only it got the first 500 tokens of the prompt, and for prompt and response it received the first 100 tokens of the prompt, a separator token and the first 500 tokens of the response all together. Each of the limits is to make sure that no portion of the training data overpowers the rest.

3. Results

Research Question 1 Answer: We can identify which LLM generates a response.

Using only the response, the CNN can identify with 85.6% accuracy, which is clearly much higher than the 33.3% chance level, and the RNN can identify with 69.3% accuracy. This means that we can identify online if something was generated by an LLM that we have a test set for. We may not be able to detect something that was generated with a private or personal model that we cannot test.

	CNN	RNN
Response only (RQ1)	85.6	69.3
Prompt only (RQ2)	33.3	33.3
Prompt and Response (RQ2)	83.1	62.7



I. Can the LLM be predicted from the prompt before seeing the response?

No, with only the prompt both models score exactly 33.3%, which is chance. This makes sense because the prompt has nothing to do with the model, it is just what the user came up with. Also each prompt was given to each of the LLMs in our training so the model that we created should have an identical understanding of each of the LLMs.

II. Does including the input improve classification?

No, adding the prompt lowered accuracy for both models because more the data that they are being trained on is identical, this is the same reason as before. The CNN model went from 85.6% to 83.1%, and the RNN went from 69.3% to 62.7%. This shows that the inclusion of the prompt is harmful to the training process.

III. If input-only works surprisingly well, why?

It does not, and it could not unless you gave each of the LLMs different prompts that each followed a pattern.

4. In-Depth Analyses

The CNN model trained on response alone returned the following confusion matrix:

	Predicted ChatGPT	Predicted Claude	Predicted Gemini	
Truly ChatGPT	197	12	41	78.8%
Truly Claude	10	222	18	88.8%
Truly Gemini	19	8	223	89.2%

56% of all the misclassifications are ChatGPT and Gemini which means that to the model they are most similar. ChatGPT is the hardest to identify but all have a quite good accuracy percentage.

The RNN model trained on response alone returned the following confusion matrix:

	Predicted ChatGPT	Predicted Claude	Predicted Gemini	
Truly ChatGPT	172	40	38	68.8%
Truly Claude	18	225	7	90.0%
Truly Gemini	86	41	123	49.2%

Using the RNN model drops the accuracy of the ChatGPT and Gemini predictions while increasing the Claude accuracy.

All the classifiers that are not just chance:

	ChatGPT	Claude	Gemini
CNN, output only	78.8	88.8	89.2
RNN, output only	68.8	90.0	49.2
CNN, input + output	88.4	88.0	72.8
RNN, input + output	33.2	74.8	80.0

No LLM is the easiest in every classifier, but Claude consistently scores well. Along with the fact that ChatGPT and Gemini have a very high rate of confusion between the two it is safe to say that Claude's responses are more distinct and easily classifiable.

Adding the prompt changed the models' overall accuracy only marginally, but the last table shows that it changed which of the LLMs the classifiers got correct.

For the CNN, accuracy increased from 78.8% to 88.4% for ChatGPT while it decreased for Gemini from 89.2% to 72.8%.

The RNN had a larger shift and was in the opposite direction. ChatGPT accuracy decreased from 68.8% to 33.2%, while Gemini accuracy increased from 49.2% to 80.0%. While this is surprising because once again the prompt gives no information about the model, these changes could insinuate that the prompts themselves happen to be in the "style" of one of the LLMs.

In my model the CNN is more accurate than the RNN both in the response and prompt plus response categories. The gaps (16.3 and 20.4 points) are more than the sampling error from the size of the test set. A reason could be that the RNN has to retain info across hundreds of layers, which is harder to train than a CNN that only has to look at three tokens at a time and keeps the strongest signal from the text.

5. Lessons and Experience

- When comparing between different transformative functions, one must make sure that the inputs to all of them are the same so that what is learned by the model is just the differences between the outputs.
- It is much easier to make sure that each of the outputs from the function are kept together either in the test or train set if you split them by correlating input instead of output.
- Make sure to make the results repeatable with a seed so if you would like to add a type of analysis but keep everything else the same, you can.
- Adding more inputs can be harmful if what you want is to identify differences. In this case the Prompts were actively harming the model because they made the data too similar to reliably tell apart.
- Sometimes a simpler model can be better than a convoluted one. The RNN would take substantially longer to train and set up but would produce worse results than the CNN.

References

Sun, M., Yin, Y., Xu, Z., Kolter, J. Z., and Liu, Z. (2025). Idiosyncrasies in Large Language Models. ICML 2025. Dataset: github.com/locuslab/llm-idiosyncrasies.